



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN
IIC2223 - TEORÍA DE AUTÓMATAS Y LENGUAJES FORMALES
PROFESOR: CRISTIAN RIVEROS
AYUDANTE: SANTIAGO MARCANO

Ayudantía 6

Minimización de Autómatas y Teorema de Myhill-Nerode

Ejercicio 1

Sea

$$A = (Q, \Sigma, \vdash, \dashv, \delta, q_0, q_f)$$

un 2DFA y sea $u \in \Sigma^*$. Suponga que se conoce la función

$$T_u : Q \cup \{\bullet\} \rightarrow Q \cup \{\perp\}.$$

Dados un estado $p \in Q$ y una letra $a \in \Sigma$, escriba un algoritmo que calcule $T_{ua}(p)$ utilizando únicamente la función de transición δ , la función T_u , el estado p y la letra a .

Su algoritmo debe retornar $\perp$ si, partiendo desde la configuración correspondiente, el autómata nunca cruza hacia la derecha de la palabra ua .

Explique la correctitud de su algoritmo y analice su complejidad.

Solución

La idea es observar qué puede ocurrir cuando el 2DFA se encuentra sobre la última letra a de ua .

Si el autómata se mueve hacia la derecha, entonces cruza inmediatamente el borde derecho de ua . En cambio, si se mueve hacia la izquierda, entra al bloque u . En este último caso, podemos utilizar T_u para determinar directamente con qué estado volverá a cruzar el borde derecho de u , sin simular nuevamente toda su ejecución sobre u .

Algoritmo 1 *

COMPUTE-T(A, T_u, p, a)

```
1: current  $\leftarrow p$ 
2: visited  $\leftarrow \emptyset$ 
3: while current  $\notin$  visited do
4:   visited  $\leftarrow$  visited  $\cup$  {current}
5:   if  $\delta(\text{current}, a)$  no está definida then return  $\perp$ 
6:   end if
7:    $(q, d) \leftarrow \delta(\text{current}, a)$ 
8:   if  $d = \rightarrow$  then return  $q$ 
9:   end if
10:  current  $\leftarrow T_u(q)$ 
11:  if current =  $\perp$  then return  $\perp$ 
12:  end if
13: end while
    return  $\perp$ 
```

Para demostrar la correctitud consideramos el siguiente invariante.

Invariante. Al comienzo de cada iteración del ciclo, si **current** = r , entonces la ejecución real del 2DFA que comienza en la configuración

$$(p, |ua|)$$

puede alcanzar la configuración

$$(r, |ua|),$$

Esto significa que cada iteración del algoritmo representa un momento en que el 2DFA, después de partir su ejecución en el estado p y con el cabezal en la última letra a de ua , se encuentra nuevamente sobre la última letra a de ua y el valor **current** indica exactamente el estado en que se encuentra el autómata en ese momento.

Demostraremos el invariante por inducción sobre el número de iteraciones.

Caso base. Antes de comenzar la primera iteración tenemos

$$\text{current} = p.$$

La ejecución comienza precisamente en la configuración

$$(p, |ua|),$$

por lo que el autómata se encuentra sobre la última letra a de ua en estado p . Además, aún no se ha realizado ninguna transición, por lo que claramente no se ha cruzado el borde derecho de ua .

Por lo tanto, el invariante se cumple antes de la primera iteración.

Paso inductivo. Supongamos que el invariante se cumple al comienzo de una iteración k .

Sea

$$\text{current} = r.$$

Por hipótesis inductiva, la ejecución real del 2DFA ha alcanzado la configuración

$$(r, |ua|),$$

sin haber cruzado todavía el borde derecho de ua .

Consideremos la transición de A sobre la letra a .

Si

$$\delta(r, a) = (q, \rightarrow),$$

el algoritmo termina en esta iteración, por lo que no es necesario demostrar el invariante para una siguiente iteración.

Supongamos entonces que

$$\delta(r, a) = (q, \leftarrow).$$

Al realizar esta transición, el autómata pasa desde la última letra a de ua hacia la última posición de u , quedando en la configuración

$$(q, |u|).$$

Si $T_u(q) = \perp$, el algoritmo también termina, por lo que nuevamente no existe una siguiente iteración que considerar.

Supongamos, finalmente, que

$$T_u(q) = s$$

para algún $s \in Q$. Por definición de T_u , desde la configuración $(q, |u|)$ la ejecución cruza el borde derecho de u en la configuración

$$(s, |u| + 1).$$

Pero $|u| + 1 = |ua|$, por lo que esta configuración es precisamente

$$(s, |ua|),$$

es decir, el autómata vuelve a encontrarse sobre la letra a .

El algoritmo realiza la asignación

$$\mathbf{current} \leftarrow T_u(q) = s.$$

Por lo tanto, al comienzo de la siguiente iteración, $\mathbf{current} = s$ coincide exactamente con el estado en que se encuentra la ejecución real sobre la última letra a .

Así, el invariante se mantiene.

Correctitud del algoritmo

Usaremos ahora el invariante para demostrar que el valor retornado por el algoritmo es exactamente $T_{ua}(p)$.

Existen tres formas en que el algoritmo puede terminar.

Caso 1: el algoritmo retorna un estado q .

Esto ocurre cuando, para el valor actual de $\mathbf{current}$,

$$\delta(\mathbf{current}, a) = (q, \rightarrow).$$

Por el invariante, en ese momento la ejecución real del 2DFA se encuentra en la configuración

$$(\mathbf{current}, |ua|)$$

y aún no ha cruzado el borde derecho de ua .

La transición anterior mueve la cabeza una posición hacia la derecha, alcanzando

$$(q, |ua| + 1).$$

Por lo tanto, la ejecución cruza el borde derecho de ua en estado q . Por definición,

$$T_{ua}(p) = q.$$

Así, el valor retornado es correcto.

Caso 2: el algoritmo retorna $\perp$ porque la ejecución no puede volver desde u .

Esto puede ocurrir si la transición sobre a no está definida, o bien si

$$\delta(\text{current}, a) = (q, \leftarrow)$$

y

$$T_u(q) = \perp.$$

Por el invariante, el autómata se encuentra realmente sobre la letra a .

En el primer caso, la ejecución se detiene sin cruzar el borde derecho de ua .

En el segundo caso, el autómata entra a u en estado q y, como $T_u(q) = \perp$, nunca vuelve a cruzar el borde derecho de u . En particular, nunca vuelve a la letra a y por lo tanto tampoco puede cruzar el borde derecho de ua .

En ambos casos,

$$T_{ua}(p) = \perp,$$

por lo que el algoritmo retorna correctamente.

Caso 3: el algoritmo encuentra nuevamente un estado visitado.

Supongamos que al comienzo de dos iteraciones distintas el valor de **current** es el mismo estado r .

Por el invariante, en ambas ocasiones la ejecución real se encuentra en la misma configuración

$$(r, |ua|).$$

Como A es determinista, a partir de una misma configuración la ejecución futura es siempre la misma. Por lo tanto, desde la segunda aparición de $(r, |ua|)$ se repetirá exactamente el mismo comportamiento que ocurrió desde la primera aparición.

Además, entre ambas apariciones el autómata no cruzó el borde derecho de ua , pues de haberlo hecho el algoritmo habría terminado en el Caso 1.

Por lo tanto, la ejecución entra en un ciclo y nunca cruzará el borde derecho de ua . Así,

$$T_{ua}(p) = \perp.$$

El algoritmo retorna entonces correctamente $\perp$.

Como estas son todas las formas posibles de terminar, concluimos que el algoritmo retorna exactamente $T_{ua}(p)$.

Ejercicio 2

Considere el siguiente problema:

Problema: ACCEPTED-PREFIXES-NFA

Input: Un NFA $A = (Q, \Sigma, \Delta, I, F)$ y una palabra $w = a_1 \cdots a_n \in \Sigma^*$.

Output:

$$\{i \in \{0, \dots, n\} \mid a_1 \cdots a_i \in L(A)\}.$$

El problema ACCEPTED-PREFIXES-NFA consiste en retornar todas las posiciones i tales que el prefijo $a_1 \cdots a_i$ de w es aceptado por A . Para $i = 0$, el prefijo corresponde a la palabra vacía ε .

Escriba un algoritmo que resuelva ACCEPTED-PREFIXES-NFA. Demuestre la correctitud de su algoritmo utilizando un invariante y analice su complejidad.

Solución

Utilizaremos como base el algoritmo de evaluación on-the-fly para NFAs.

El conjunto S contendrá los estados en los que puede encontrarse el autómata después de procesar el prefijo leído hasta el momento.

Algoritmo 2 ACCEPTED-PREFIXES-NFA(A, w)

```

1:  $S \leftarrow I$ 
2:  $Answer \leftarrow \emptyset$ 
3: if  $S \cap F \neq \emptyset$  then
4:    $Answer \leftarrow Answer \cup \{0\}$ 
5: end if
6: for  $i = 1$  to  $n$  do
7:    $S_{old} \leftarrow S$ 
8:    $S \leftarrow \emptyset$ 
9:   for all  $p \in S_{old}$  do
10:     $S \leftarrow S \cup \{q \mid (p, a_i, q) \in \Delta\}$ 
11:   end for
12:   if  $S \cap F \neq \emptyset$  then
13:     $Answer \leftarrow Answer \cup \{i\}$ 
14:   end if
15: end for
16: return  $Answer$ 

```

Para demostrar la correctitud del algoritmo consideramos el siguiente invariante.

Invariante. Después de procesar las primeras i letras de la palabra, el conjunto S contiene exactamente lo siguiente:

$$S = \{q \in Q \mid \text{existe una ejecución de } A \text{ sobre } a_1 \cdots a_i \text{ que termina en } q\}.$$

Demostraremos el invariante por inducción sobre i .

Caso base ($i = 0$). Antes de comenzar el ciclo todavía no se ha leído ninguna letra de w , por lo que el prefijo procesado corresponde a ε .

El algoritmo inicializa $S \leftarrow I$ que es precisamente el conjunto de estados alcanzables mediante una ejecución sobre ε , por lo que el invariante se cumple para $i = 0$.

Paso inductivo. Supongamos que el invariante se cumple después de procesar las primeras $i - 1$ letras, es decir,

$$S = \{p \in Q \mid \text{existe una ejecución sobre } a_1 \cdots a_{i-1} \text{ que termina en } p\}.$$

Al comenzar la iteración i , el algoritmo realiza $S_{\text{old}} \leftarrow S$ y luego $S \leftarrow \emptyset$.

Por la hipótesis inductiva, S_{old} contiene exactamente los estados alcanzables después de leer $a_1 \cdots a_{i-1}$.

Luego, para cada $p \in S_{\text{old}}$, el algoritmo agrega a S todos los estados q tales que

$$(p, a_i, q) \in \Delta.$$

Por hipótesis inductiva, existe una ejecución sobre $a_1 \cdots a_{i-1}$ que termina en p . Agregando la transición (p, a_i, q) obtenemos una ejecución sobre

$$a_1 \cdots a_{i-1} a_i$$

que termina en q . Por lo tanto, todo estado agregado a S es alcanzable después de leer $a_1 \cdots a_i$.

Por lo tanto, el invariante se mantiene.

Correctitud del algoritmo

Usaremos ahora el invariante para demostrar que el conjunto *Answer* retornado por el algoritmo es exactamente

$$\{i \in \{0, \dots, n\} \mid a_1 \cdots a_i \in L(A)\}.$$

Por el invariante, después de procesar las primeras i letras, S contiene exactamente los estados en los que puede terminar una ejecución de A sobre el prefijo $a_1 \cdots a_i$.

Por definición de aceptación de un NFA,

$$a_1 \cdots a_i \in L(A)$$

si, y solo si, existe una ejecución sobre dicho prefijo que termina en algún estado final.

Utilizando el invariante, esto ocurre si, y solo si,

$$S \cap F \neq \emptyset.$$

El algoritmo comprueba precisamente esta condición después de procesar cada prefijo. Si

$$S \cap F \neq \emptyset,$$

agrega i a *Answer*.

El caso $i = 0$ también se revisa antes de comenzar el ciclo. En ese momento $S = I$, por lo que

$$S \cap F \neq \emptyset$$

si, y solo si, existe un estado que es a la vez inicial y final, lo cual equivale a

$$\varepsilon \in L(A).$$

Por lo tanto, para todo $i \in \{0, \dots, n\}$,

$$i \in \text{Answer} \iff a_1 \cdots a_i \in L(A).$$

Así, al finalizar, el algoritmo retorna exactamente el conjunto de posiciones correspondientes a los prefijos de w aceptados por A .

Ejercicio 3

Considere el siguiente problema:

Problema: LONGEST-SUBSTRING-NFA

Input: Un NFA $A = (Q, \Sigma, \Delta, I, F)$ y una palabra $w = a_1 \cdots a_n \in \Sigma^*$.

Output: Un par de índices (i, j) , con $1 \leq i \leq j \leq n$, tal que $a_i \cdots a_j \in L(A)$ y $j - i + 1$ sea máximo entre todas las subpalabras de w aceptadas por A .

El problema LONGEST-SUBSTRING-NFA consiste en encontrar una subpalabra contigua de w de largo máximo que sea aceptada por el NFA A . Si ninguna subpalabra no vacía de w pertenece a $L(A)$, retornar NONE.

Escriba un algoritmo *on-the-fly* que resuelva LONGEST-SUBSTRING-NFA en tiempo $O(|A| \cdot |w|)$. Demuestre la correctitud de su algoritmo utilizando un invariante y analice su complejidad.

Solución

Proxima ayudantia :)